A Complete Guide To gRPC

Section 1: Introduction

1. Topics:
    1. Why should we learn gRPC?
    2. What kind of problems does it solve?
    3. What are the benefits of gRPC?
2. Use-case: Monolithic Application
    1. They have modular architecture
        1. But the application is packaged and deployed as one single unit
    2. Modules:
        1. All communications among modules are done using internal calls
            1. Very fast
            2. Easy to manage
            3. ...
    3. Disadvantages:
        1. If application becomes too big:
            1. Hard to manage
            2. A module might take too much memory and kill the entire application eventually
            3. ...
3. Solution:
    1. Micro-service based architecture
        1. We split big monolith into multiple services
            1. Based on sub-domains
        2. The services talk to one-another to fulfill an incoming request
            1. The services are designed by exchanging JSON over HTTP
                1. It became the standard
                    1. Disadvantages:
                        1. Problem 1: Request & Response Protocol
                            1. HTTP 1.1 - One-to-one
                                1. 3 message exchanges to establish a TCP connection
                                2. We then send a request and get a response
                                    1. We need to wait for the response to return, before sending another request via the same connection
                                        1. We may need another connection to send in parallel
                                            1. Problem: It takes a significant amount of time
                        2. Problem 2: Headers
                            1. HTTP is stateless protocol
                                1. Headers are sent in every request
                                2. Each request carries info like cookies
                                3. Plain text - relatively large in size
                                4. Can not be compressed
                        3. Problem 3: Serialization & Deserialization
                            1. If person class object needs to be sent to another service:
                                1. It is serialized
                                2. It is sent over the network
                                3. It is de-serialized into a Person object (of it's type)
                            2. Machines understand binary formats
                            3. Disadvantage: Text formats are relatively large in size and takes a lot of time, cpu, memory to send and process
                                1. It happens for every single call
                        4. Problem-4: API Contract
                            1. There is no strict typing, contract when it comes to JSON
                                1. Producer might be sending info in one format
                                2. Consumer might send info in a different format
                            2. It is not a standard
                            3. A solution:
                                1. Developer provides a client library for his service
                                    1. Developer can add it as a dependency
                        5. Problem-5: Client SDK
                            1. Providing client libraries for the service
                                1. Problem: If there are multiple clients using multiple languages
                                    1. Java
                                    2. JS
                                    3. C++
                                    4. Ruby
                                    5. Python
                                    6. Go
                                    7. ...
                                        1. Do we need to write client libraries for all the languages?

Problem Statement

1. Stubby:
    1. RPC Framework from Google
        1. For inter-service communication
    2. 15 years
    3. 10 billion reqs/sec!!!
    4. Cross-platform
        1. Multiple programming languages are used
    5. Problem: Tightly coupled with infrastructure
2. gRPC - it is a framework for inter-services communication
    1. Developed at Google
        1. A complete re-write
    2. Inspired by Stubby

3. Released in 2016
4. Adopted by
    1. Netflix
    2. Microsoft
    3. ...
5. Belongs to CNCF
    1. Cloud Native Computing Foundation
        1. Opensource home for cloud native applications
            1. k8s
    2. Google engineers are contributing a lot to the project
        1. Leave a star

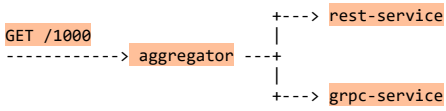
HTTP/2-vs-HTTP/1.1

1. How does gRPC address the problems?
    1. HTTP 1.1 (1997)
        1. It works on top of TCP
            1. TCP - 3 way handshake process
                1. Significant amount of time is spent in establishing a connection
                    1. 3 messages are exchanged to form a stable connection
                    2. Client then sends a request and gets a response
                    3. Client waits for response before sending another request
                2. It was okay before, because applications were static
                    1. We have multiple urls to invoke:
                        1. JS files are downloaded
                        2. Images are downloaded
                        3. Other files are downloaded
                        4. ...
                    2. Problem:
                        1. If we send requests sequentially, the page takes a long time to get rendered
        2. Open the url:
            1. Many images - a grid of 180 images
                1. Chrome has to send 180 requests
                    1. Network section
                        1. Many requests are sent - 7.5 seconds
                        2. 6 TCP connections are maintained
                            1. All requests are not sent on a single connection (parallelism)
                2. Change to http2
                    1. 545 ms
                        1. Only 1 TCP connection
2. HTTP 1.1:
    1. Browser sends a request and waits for response to return
        1. Browser sends 6 requests over 6 TCP connections & waits
        2. The connections are reused to send other requests after responses are returned respectively
3. HTTP 2:
    1. Enables multiplexing
        1. One connection is enough to send multiple parallel requests
            1. We do not have to wait for response to come back
                1. CSS, JS, HTML, Images, ...
            2. One TCP connection is enough
    2. It is:
        1. a binary protocol
            1. We have two frames: (relatively small in size)
                1. Headers frame
                2. Data frame
        2. Header compression
            1. Example:
                1. Request #1:
                    1. :method: GET
                    2. :scheme: https
                    3. :host: example.com
                    4. :path: /resource
                    5. accept: image/jpeg
                    6. user-agent: Mozilla/5.0 ...
                2. Headers frame (Stream 1)
                    1. :method: GET
                    2. :scheme: https
                    3. :host: example.com
                    4. :path: /resource
                    5. accept: image/jpeg
                    6. user-agent: Mozilla/5.0 ...
                3. Request #2:
                    1. Headers frame (Stream 3) (info is not repeated - only changes are sent)
                        1. :path: /new_resource
        3. Flow control
            1. Back-pressure
                1. Sender will not send too much information that the receiver cannot handle
                    1. We can send request and response as streaming data
        4. Multiplexing
            1. One connection and multiple parallel requests and get the responses
4. gRPC - Benefits
    1. HTTP2 is default (gRPC uses it but it can use any protocol - inherits benefits of HTTP2)
        1. Binary
        2. Multiplexing
        3. Flow-control
    2. Non-blocking, Streaming bindings

3. Protobuf: (replacing JSON)
    1. Strict Typing
    2. DTO
    3. Service definitions
    4. Language-agnostic
        1. Client libraries can be provided without doing anything
    5. Auto-generated bindings for multiple languages
4. Great for mobile apps
    1. Avoids
        1. Serialization and deserialization
        2. Blocking requests and responses

gRPC-vs-REST - Performance Comparison

1. gRPC vs REST
    1. REST
        1. Architectural style
        2. Resource oriented
            1. Example: Book
                1. HTTP does CRUD
        3. ~JSON + HTTP
            1. It became standard because:
                1. Easy to develop and test because of tooling
                2. Extended browser support
    2. gRPC
        1. A RPC framework
            1. It comes with all the tools to make inter-microservice communication effective and efficient
        2. More flexible & Action oriented
        3. Specific to inter-services communication
2. gRPC vs REST



    1. Client wants to collect squares of numbers upto 1000
    2. The two service gives square only for the given number
        1. Aggregator: Sends all the numbers the client wants (1-1000) and collect all the results and respond back to the client
    3. Example to simulate multiple IO requests
    4. Aggregator design:
        1. It chooses one of the services to process the aggregated requests

```
// REST
rest/unary/1000

// gRPC
grpc/unary/1000
grpc/stream/1000
```

            1. gRPC gives bidirectional streaming
            2. REST is only unary
        2. Apache Bench tool need to be used to benchmark

```
ab -n 1000 -c 100 localhost:8080/rest/unary/1000
```

            1. 1000 - requests
            2. 100 - concurrent users
            3. Results:
                1. ~ 87 s
                2. 11.5 requests /s
                3. Response time: 9.5 s per user
        3. gRPC unary:

```
ab -n 1000 -c 100 localhost:8080/grpc/unary/1000
```

            1. ~ 27 requests/ s
            2. 36.19 s
            3. Response time: 4.1 s per user
        4. gRPC stream:

```
ab -n 1000 -c 100 localhost:8080/grpc/stream/1000
```

            1. ~ 121 requests/ s
            2. 8.2 s
            3. Response time: < 1 s per user (for 100 users)
        5. User experience from Browser:
            1. Unary: 10,000 - 5 seconds

2. gRPC stream: 10,000 - < 1 second

Course Structure

1. Topics:
    1. Protocol Buffers/ protobuf
    2. gRPC
        1. Client/Server application
        2. Different types of RPC
        3. Client/Server - Game assignment
        4. Load Balancing
        5. Authentication/Metadata/Contexts
        6. Error Handling
        7. Deadline
        8. Intercepting the requests
    3. Bonus: gRPC - Spring Boot Integration
2. Note:
    1. Assumption Experience with Java 8+
    2. Pace could be slow or fast depending on your experience
        1. But You would be comfortable with gRPC
    3. Would like to show things from scratch

Course Resources Download

1. Github Repo
2. Slides

Section 2: Protocol Buffers

1. Introduction
    1. IDL (Interface Description Language) for API
    2. Platform neutral
    3. Language neutral
    4. Serializing/Deserializing structured data
    5. Very Fast/Optimized for interservices communication
    6. Provides client libraries automatically for many languages
        1. Java
        2. C++
        3. Javascript
        4. Go
        5. Ruby
        6. C#
        7. Python

Proto - Introduction

1. Protobuf:
    1. It is an IDL for APIs
        1. Like WSDL for SOAP
        2. It is a mechanism for serializing and deserializing structured data
            1. It is very fast and optimized for inter-services communication
2. Example:
    1. DTO:

```
public class Person {
    private String name;
    private int age;

    // getters
    // setters
}
```

    2. Protobuf file:

```
message Person {
    string name = 1;
    int32 age = 2;
}
```

[Resource] - Proto Dependencies

1. Newer version: Github - pom.xml

Proto - Project Setup

1. Maven project
    1. New project
        1. groupId: com.guru
        2. Dependencies:
            1. Download the pom.xml
        3. Plugin
            1. Reads proto file and converts it into java classes during the compile phase
            2. Source directory: `src/main/proto` **(M)**
                1. Right click on project structure
                    1. Modules

1. Treat it as a source directory
3. Settings:
    1. Plugins
        1. Protocol Buffer Editor

Proto - A Simple Person Message Creation

1. `src/main/proto`
    1. person.proto

```
syntax = "proto3";

message Person {
    string name = 1;
    int32 age = 2;
}
```

    1. Version 1 - Internal to Google
    2. Version 2 - Explicitly mention if it is optional or mandatory
    3. Version 3 - All are required fields
    4. Compile: `mvn clean compile`
        1. `generated-sources`
            1. `protobuf`
                1. `java`
                    1. `PersonOuterClass`
        2. There are options to work with the generated classes

Proto - Java Options

1. A Person class is created inside an outer class
2. person.proto

```
syntax = "proto3"

option java_multiple_files = true;

...
```

    1. Extracts Person class as a separate file
    2. Problem: Class is created without a package
        1. Solution:

```
option java_package = "com.vinsguru.models";
```

        2. Compile
            1. Person is inside package

Proto - Person Builder Pattern Demo

1. Person class is final class
2. Person constructor is private
    1. Solution: Use `Person.newBuilder()` to construct
3. `src/main/java/protobuf/PersonDemo.java`

```
public class PersonDemo {
    public static void main(String[] args) {
        Person sam = Person.newBuilder()
                        .setName("sam")
                        .setAge(10)
                        .build();
        System.out.println(sam);
    }
}
```

Proto - Equals Method

1. `newBuilder()` instantiates a new object
2. Example:

```
Person sam2 = Person.newBuilder()
                .setName("sam")
                .setAge(24)
                .build();

System.out.println(sam1.equals(sam2));
```

    1. We cannot override `.equals` method
        1. If we have to compare using custom method (insensitive compare say), we need to construct a utility class that accepts both the objects

Proto - Serialization & Deserialization

1. Writing to a file and read from the file
2. Example:

```
Path path = Paths.get("sam.ser"); // **(M)**
Files.write(path, sam.toByteArray());

//...

byte[] bytes = Files.readAllBytes(path);
Person newSam = Person.parseFrom(bytes); // **(M)**
System.out.println(newSam);
```

[Resource] - Jackson Dependency

1. Add the following dependency: https://github.com/vinsguru/grpc-java-course/blob/master/protobuf/pom.xml#L28-L32

Proto vs JSON - Performance Comparison - Part - 1

1. Performance test between Protobof and JSON
2. Add Jackson dependency
3. `com.vinsguru.json.JPerson.java`

```
public class JPerson {
    private String name;
    private int ag;

    // getters and setters
}
```

4. `protobuf.PerformanceTest.java`

```
public static void main(String[] args) {
    // json
    Runnable runnable;

    // protobuf
    Runnable runnable2;
}

private static void runPerformanceTest(Runnable runnable, String method) {
    long time1 = System.currentTimeMillis();
    for (int i = 0; i < 1_000_000; i++) {
        runnable.run();
    }
    long time2 = System.currentTimeMillis();
    System.out.println(method + ": " + (time2 - time1) + " ms");
}
```

Proto vs JSON - Performance Comparison - Part - 2

1. Code:

```
// json
JPerson person = new JPerson();
person.setName("sam");
person.setAge(10);
ObjectMapper mapper = new ObjectMapper();
Runnable json = () -> {
    try {
        byte[] bytes = mapper.writeValueAsBytes(person);
        JPerson person1 = mapper.readValue(bytes, JPerson.class);
    } catch (Exception e) {
        e.printStackTrace();
    }
};

Person sam = Person.newBuilder()
                .setName("sam")
                .setAge(10)
                .build();
Runnable proto = () -> {
    try {
        byte[] bytes = sam.toByteArray();
        Person sam1 = Person.parseFrom(bytes);
    } catch (InvalidProtocolBufferException e) {
        e.printStackTrace();
    }
};

runPerformanceTest(json, "JSON");
runPerformanceTest(proto, "PROTO");

// JSON: 1409 ms
// PROTO: 197 ms
```

Proto vs JSON - Performance Comparison - Part - 3

1. Running a few times:

```
for (int i = 0; i < 5; i++) {
    runPerformanceTest(json, "JSON"); // change to 5 M
    runPerformanceTest(proto, "PROTO");
}
```

    1. Run:
        1. Proto is around 465 (~ 0.5 s)
        2. JSON is around 3250 (~ 3.25 s)

Proto - Adding Comments

1. Comments: Same as Java

```
/*
    This is my person proto file
*/

// name field
```

Note: Known Issue with JS

1. Generating JS files from proto
    1. It is broken in the recent version
        1. Download vertion 3.19.0 for the lecture

Proto - Auto Generate JS File - Demo

1. Creation of JS files
2. Terminal:
    1. `cd src/main/proto`
    2. `../../../target/protoc-plugins/protoc-3.6.1-linux-x64_64.exe --js_out=./ *.proto` **(M)** (all proto files or some specific proto files)
        1. `person.js` is created

[Optional]-[Resource] - Protoc

1. protoc

Protoc Tool Installation

1. Installation
    1. Github protobuf project
        1. Tags
            1. Releases
                1. `protoc-<os>..`
                2. Unzip
                3. Keep it in path
                4. `protoc`
2. From command-line:

```
protoc --js_out=./ person.proto # **(M)**
```

Proto - Scaler Types

1. Types:
    1. Java Type | Proto Type
        1. `int` | int32 **(M)**
        2. `long` | int64 **(M)**
        3. `float` | float
        4. `double` | double
        5. `boolean` | bool **(M)**
        6. `String` | string // it is primitive type in Proto
        7. `byte[]` | bytes **(M)**
2. Proto - Composition
    1. Modular design for better code re-usability
        1. We can include types inside other types:

```
message Car {
    string make = 1;
    string model = 2;
    int32 year = 3;
}

message Address {
    int32 postbox = 1;
    string street = 2;
    string city = 3;
}

message Person {
    string name = 1;
    int32 age = 2;
    Car car = 3;
    Address address = 4;
}
```

Proto - Composition Demo

1. person.proto - we can have multiple types

```
syntax = "proto3";

option java_multiple_files = true;
option java_package = "com.vinsguru.models";

message Address {
    int32 postbox = 1;
    string street = 2;
```

```
    string city = 3;
}

message Car {
    string make = 1;
    string model = 2;
    int32 year = 3;
}

message Person {
    string name = 1;
    int32 age = 2;
    Address address = 3;
    Car car = 4;
}
```

1. Build
2. New class: `com.vinsguru.protobuf.CompositionDemo.java`

```
public static void main(String[] args) {
    Address address = Address.newBuilder()
            .setPostbox(123)
            .setStreet("main street")
            .setCity("Atlanta")
            .build();

    Car car = Car.newBuilder()
            .setMake("Honda")
            .setModel("Accord")
            .setYear(2020)
            .build();

    Person person = Person.newBuilder()
            .setName("sam")
            .setAge(25)
            .setCar(car)
            .setAddress(address)
            .build();

    System.out.println(sam);
}
```

Proto - Collection Demo

1. Types:
    1. Java Type | Proto Type
        1. Collection / List | repeated **(M)**
        2. Map | map **(M)**
2. person.proto

```
message Person {
    ...
    repeated Car car = 4;
    ...
}
```

3. Class:

```
Car car = Car.newBuilder()
                .setMake("Honda")
                .setModel("Accord")
                .setYear(2020)
                .build();

Car car = Car.newBuilder()
                .setMake("Honda")
                .setModel("Accord")
                .setYear(2020)
                .build();

Person sam = Person.newBuilder()
                .setName("sam")
                .setAge(25)
                .addCar(accord)
                .addCar(civic)
                ...
```

    1. Adding a list of cars

```
List<Car> cars = new ArrayList<>();

cars.add(accord);
cars.add(civic);

Person sam = Person.newBuilder()
                ...
                .addAllCars(cars)
                ...
```

Proto - Map Demo

1. person.proto

```
message Dealer {
    map<int32, Car> model = 1;
}
```

2. New class: MapDemo

```
Car accord = ...
Car civic = ...

Dealer dealer = Dealer.newBuilder()
    .putModel(2014, civic)
    .putModel(2020, accord)
    .build();

Dealer dealer = Dealer.newBuilder()
    .putAllModels(models)
    .build();

dealer.getModelCount(); // size of the map - **(M)**
dealer.getModelOrThrow(2014); // returns object
dealer.getModelOrThrow(2000); // throws exception - IllegalArgumentException
dealer.getModelOrDefault(2019, accord); // return object if it exists or default object

dealter.getModelMap(); // returns Map
```

Proto - Enum Demo

1. person.proto

```
message Car {
    ...
    BodyStyle body_style = 4; // preferred style
}

enum BodyStyle {
    SEDAN = 0;
    COUPE = 1;
    SUV = 2;
}
```

2. Class:

```
dealer.getModelOrThrow(2014)
    .getBodyStyle(); // SEDAN is default (first value)

// Next

...
.setBodyStyle(BodyStyle.SEDAN)
...
```

Proto - Default Values

1. DefaultValueDemo.java

```
public static void main(String[] args) {
    Person person = Person.newBuilder().build();

    System.out.println("City : " + person.getAddress().getCity()); // Doesn't throw null-pointer exception
}
```

    1. Why?
        1. No null in protobuf
            1. Problem: No difference between explicitly assigning empty or was it assigned by default
                1. Solution:

```
person.hasAddress(); // has the address been set?
```

2. Default values:

    1. Proto Type | Default
        1. int32 / any number type | 0
        2. bool | false
        3. string | empty string
        4. enum | first value
            1. 0th value
        5. repeated | empty list
        6. map | wrapper / empty map

3. DefaultValueDemo.java

    1. There is no: `hasName` or `hasAge`
        1. It is generated for other types (not scalar types)

Proto - Importing Modules

1. We can construct re-usable modules using protobuf
    1. Proto files can be packaged & imported
        1. Packaging - similar to jar or war
2. To keep organized:
    1. `src/main/proto`
        1. New package: `common`
            1. New proto file: `car.proto`

```
syntax = "proto3";
```

```
package common;

option java_multiple_files = true;
option java_package = "com.vinsguru.models";

message Car {
    ...
}

enum BodyStyle {
    ...
}
```

2. New proto file: `address.proto`
    1. Copy paste address details
3. New proto file: `dealer.proto`
    1. Copy paste dealer details

2. person.proto

```
import "common/address.proto"; // **(M)**
import "common/car.proto";

message Person {
    ...
    common.Address address = 3;
    common.Car car = 4;
    ...
}
```

3. dealer.proto

```
import "common/car.proto";

message Dealer {
    map<int32, common.Car> model = 1;
}
```

4. Compile and run

Proto - OneOf - Demo

1. Java - Interface / Implementations
    1. Car (interface/base class)
        1. SportsCar
        2. LuxuryCar
2. Proto - OneOf
    1. Example: Assume that there is a method that accepts credentials
        1. Options: (give one of them but not both)
            1. Email login
            2. Phone login
    2. OneOf - means either one or the other (not both)

        1. credentials.proto

```
syntax = "proto3";

option java_multiple_files = true;
option java_package = "com.vinsguru.models";

message EmailCredentials {
    string email = 1;
    string password = 2;
}

message PhoneOTP {
    int32 number = 1;
    int32 code = 2;
}

message Credentials {
    oneOf mode {
        EmailCredentials emailMode = 1;
        PhoneOTP phoneMode = 2;
    }
}
```

            1. `Credentials` has only one of the objects

        2. Class: OneOfDemo.java

```
public static void main(String[] args) {
    EmailCredentials emailCredentials = EmailCredentials.newBuilder()
        .setEmail("nobody@gmail.com")
        .setPassword("admin123")
        .build();

    PhoneOTP phoneOTP = PhoneOTP.newBuilder()
        .setNumber(3343232323)
        .setCode(454)
        .build();

    Credentials credentials = Credentials.newBuilder()
        .setEmailMode(emailCredentials)
        .build();
```

```
        login(credentials);
    }

    private static void login(Credentials credentials) {
        System.out.println(credentials.getEmailMode());
    }
```

1. `Credentials` - It is like a generic model object (that is similar to an interface)
2. Set `phoneOTPMode` also

    ```
    .setEmailMode(emailCredentials)
    .setPhoneMode(phoneOTP)
    ```

    1. The second call will erase the first call's actions
3. How do we know which mode has been set (if there are too many)?

    ```
    switch (credentials.getModeCase()) {
        case EMAILMODE:
            System.out.println(credentials.getEmailMode());
            break;
        case PHONEMODE:
            System.out.println(credentials.getPhoneMode());
    }
    ```

    1. A convenient enum has been created
    2. We are giving them response or pass error response

Proto - Wrapper Types

1. In Java have primitive types and wrapper types
    1. Proto also have primitives and wrappers
2. Exmaple:

    ```
    import "google/protobuf/wrapper.proto"; // **(M)**

    ...
    message Person {
        ...
        google.protobuf.wrapper.Int32Value age = 2; // **(M)**
        ...
    }
    ```

    1. `hasAge` method is generated
    2. The `setAge` doesn't accept `int`

        ```
        .setAge(Int32Value.newBuilder().setValue(25).build())
        ...

        System.out.println(sam.hasAge());
        ```

Proto - How It Works - Demo

1. Performance test:
    1. Why is ProtoBuf is faster than JSON

        ```
        bytes[] bytes = mapper.writeValueAsBytes(person);
        System.out.println(bytes.length);
        ...
        bytes[] bytes = sam.toByteArray();
        System.out.println(bytes.length);

        1. Printing only 1 (not millions of times)
            1. JSON: 23 elements
            2. PROTO: 7 elements
        2. Why the difference?
        ```

Person Proto

1. JSON:

    ```
    "name":"sam",
    "age":10
    ```

2. Proto:

    ```
    1=sam
    2=10
    ```

3. The numbers are called tags
    1. The numbers can be anything
4. The names are only for developers (protobuf doesn't care)
    1. Once the data reaches the service, the service has the same protocol buffer files
        1. It will do reverse map the tags to the names (JSON uses keys instead of tags)
5. If we use any random numbers (100, 500, ...), the number of characters will be more

```
100=sam
500=10
```

1. Community recommends:
    1. 1 - 15 - 1 byte is used
        1. Use these tags for most frequently used fields
    2. 16 - 2047 - 2 bytes are used
        1. Use these tags for less frequently used fields
        2. If we don't set a value, protobuf doesn't even send empty

```
"car":[]
```

Proto - Tags

1. Field Numbers:
    1. Each field is assigned with unique number
    2. **1-15 for frequently used fields (uses 1 byte)**
    3. 16-2047 - uses 2 bytes
        1. For not frequently used fields
    4. 1 is smallest
        1. Enum - 0 (default type)
    5. 2^29 - 1
        1. Max limit (1/2 billion)
    6. 19000-19999 - reserved
        1. Pluging shows error
    7. Do not change the field number once it is in use
        1. Once clients start using
            1. It is risky

Proto - API/Message Changes - Part - 1

1. When we publish an API, we expose message formats to clients
2. Suppose we need to update proto files
    1. If some of the clients cannot upgrade to the latest version
3. How does the service communication get affected?

    1. New file: television.proto

```
syntax = "proto3";

import "common/address.proto";
import "common/car.proto";

option java_multiple_files = true;
option java_package = "com.vinsguru.models";

// v1
message Television {
    string brand = 1;
    int32 year = 2;
}
```

        1. New class: VersionCompatibilityTest

```
Television television = Television.newBuilder()
                        .setBrand("Sony")
                        .setYear(2005)
                        .build();

Path path = Paths.get("tv-v1");
Files.write(pathV1, television.toByteArray());

//
byte[] bytes = Files.readAllBytes(pathV1);
System.out.println(bytes);
```

Proto - API/Message Changes - Part - 2

1. television.proto

```
// v2
message Television {
    string brand = 1;
    int32 model = 2;
    Type type = 3;
}

enum Type {
    HD = 0;
    UHD = 1;
    OLED = 2;
}
```

    1. When we deserialize the object, the new name generated can be used (since the mapping is with tag and not with name)
        1. It is okay to rename (for existing client it works)
    2. If a new value is sent, existing client don't have to change the code
    3. V2 client serilizing code:

```
Television television = Television.newBuilder()
                            .setBrand("sony")
                            .setModel(2016)
                            .setType(Type.OLED)
                            .build();

Files.write(pathV2, television.toByteArray());
```

4. Read V2 object into V1 object

```
byte[] bytes = Files.readAllBytes(pathV2);
System.out.println(Television.parseFrom(bytes));
```

Proto - API/Message Changes - Part - 3

1. Deleting a field
    1. Remove `model` field in `Television`
        1. For v3 clients, it works fine
        2. for v2 clients, it returns the following result for non existing field (since it is in the file):

```
2: 2016
```

            1. We can ignore it

Proto - API/Message Changes - Part - 4

1. Adding new field:

```
message Television {
    ...
    int32 price = 2;
    ...
}
```

    1. Assume some clients didn't update their client
    2. Problem:

```
brand: "sony"
price: 2015
```

        1. `model` is not `price`
        2. Solution: If we remove a field, we need to specify that it is deprecated

```
message Television {
    string brand = 1;

    reserved 2;
    reserved "year", "model";

    int32 price = 2; // cannot be used because 2 is reserved (choose a different number)
    Type type = 3;
    int32 year = 4; // cannot be used
    string model = 5; // cannot be used
}
```

            1. We can use wrapper type to check if price is explicitly set or not

Proto - Summary

1. IDL (Interface Description language) for API
    1. For DTOs
    2. for APIs
2. Platform neutral
3. Language neutral
4. Serializing / Deserializing structured data
5. Very Fast / Optimized for inteservices communication
6. Provides client libraries automatically for many languages!
    1. Java
    2. C++
    3. Javascript
    4. Go
    5. Ruby
    6. C#
    7. Python
7. Guidelines
    1. 1-15 for frequently used fields
    2. Do not reorder the fields once it is in use
        1. Once client started used it
    3. Adding New fields will not break old proto
        1. If v1 client sends info to v2 server, v2 server gets default values for new fields
        2. If v2 client sends info to v1 server, v1 server ignores the new fields
    4. Removing fields will not break old proto
        1. Use reserved
            1. To avoid problems
    5. Changing Type
        1. int32 -> int64 is OK (int to long)
        2. int64 -> int32 might be a problem
            1. Number may be incorrect since we lose bits

6. Renaming is OK. But be cautious
7. Keep the proto as separate maven-module and add them as dependency in other modules

Section 3: gRPC - Introduction & Unary RPC

gRPC - Introduction

1. High-performance, open-source RPC framework
    1. It is action oriented (as opposed to resource oriented)
2. Developed at Google
3. Client app directly invokes Server method on a different machine
    1. We define a service or interface for the services we want to expose through proto file
    2. Using proto file, we will be generating abstract classes or interfaces using proto file
        1. Implemented by the server
    3. Using proto file, we can also generate client library or stubs
    4. The client library will be used to interact with the server
4. Service is defined using proto

gRPC - Synchronous vs Asynchronous

1. Client's call to the server can be Sync/Async (choice is for the client)
    1. Sync -> blocking / waiting for the response
    2. Async -> Register a listener for call back
2. It is completely up to the client
    1. It also depends on the RPC

gRPC - RPC Types

RPC - Types

1. Unary
    1. We make a request and get a response
    2. RPC is complete once we receive the response
2. Server-streaming
    1. Client sends one request and server sends one or many response (one-to-many)
    2. The streaming ordered sequence of messages
        1. Example: If we search for keyword, Amazon might send response in chunks
    3. **Use-case**: Used if client cannot handle huge response at once
        1. Example: Pagination
3. Client-streaming
    1. Client sends a stream of requests for which server sends only one response
    2. Example: File upload
        1. Client sends a file in small chunks
        2. Once file upload is complete, server sends a single acknowledgement
4. Bidirectional-streaming
    1. Streaming on both sides
    2. The streams are independent
        1. It is not that client sends one request and server sends one response each time
        2. We can design to send one response for every three requests
            1. Not one to one
    3. **Use-case**: If server and client have to work in an interactive way to finish a task
        1. Example: Ping-pong messaging

[Resource] - gRPC Dependencies

1. Dependencies to be added to a project as per: https://github.com/vinsguru/grpc-java-course/blob/master/grpc-intro/pom.xml

gRPC - Project Setup

1. New maven project
2. Add dependencies
    1. netty-shaded - required to run grpc server
    2. junit - Test cases will be used as clients
3. Add plugins
4. New `proto` directory
    1. `src/main/src`

Unary - Service Definition

1. Bank service
    1. It maintains a set of pre-defined accounts with specific balance
        1. Get method: If we send account number, we will be getting balance from the server
2. New proto file: bank-service.proto

```
syntax = "proto3";

option java_multiple_files = true;
option java_package = "com.vinsguru.models";

message BalanceCheckRequest {
    int32 account_number = 1;
}

// USD
```

```
message Balance {
    int32 amount = 1;
}

service BankService {
    rpc getBalance(BalanceCheckRequest) return (Balance);
}
```

1. Generate source code
    1. It generates abstract class
    2. It generates client libraries to interact with server

Unary - BankService - getBalance - Implementation

1. `protobuf/grpc-java/com/vinsguru/moels/BankServiceGrpc.java`
    1. Server has to override this class
2. New class: `com.vinsguru.server.BankService.java`

```
public class BankService extends BankServiceGrpc.BankServiceImplBase {
    @Override
    public void getBalance(BalanceCheckRequest request, StreamObserver<Balance> responseObserver) {
        int accountNumber = request.getAccountNumber();
        Balance balance = Balance.newBuilder()
                .setAmount(accountNumber * 10)
                .build();
        responseObserver.onNext(balance);
        responseObserver.onCompleted();
    }
}
```

1. `Balance` object is sent via `StreamObserver`
2. There are two channels
    1. Data channel
        1. Use `onNext()` if we want to use the data channel
    2. Error channel
        1. Use `onError()` if we want to use the error channel
            1. The response is closed (we don't have to call `onCompleted()`
3. `onCompleted()` - called when we have nothing else to receive
    1. The response is closed

Unary - GRPC Server

1. It is used to expose our service
2. New class: `GrpcServer.java`

```
public static void main(String[] args) throws IOException, InterruptedException {
    Server server = ServerBuilder.forPort(6565)
                    .addService(new BankService())
                    .build();
    server.start();
    server.awaitTermination(); // if this is not called, the control will exit main
}
```

Unary - Client's Blocking Stub

1. `src/test/java/com/vinsguru/client/BankClientTest.java`

```
@TestInstance(TestInstance.Lifecycle.PER_CLASS)
public class BankClientTest {

    @BeforeAll
    public void setup() {

        private BankServiceBrcp.BankServiceBLockingStub blockingStub;

        ManagedChannel managedChannel = ManagedChannelBuilder.forAddress("localhost", 6565)
            .usePlaintext() // no SSL
            .build();

        this.blockingStub = BankServiceGrpc.newBlockingStub(managedChannel); // we wait for the response
    }

    @Test
    public void balanceTest() {
        BalanceCheckRequest balanceCheckRequest = BalanceCheckRequest.newBuilder()
                .setAccountNumber(5)
                .build();
        Balance balance = this.blockingStub.getBalance(balanceCheckRequest);
        System.out.println("Received : " + balance.getAmount());
    }

}
```

1. `@TestInstance(TestInstance.Lifecycle.PER_CLASS)` - avoid creation of static variables but instance variables

BankService - A Simple DB Using Map

1. Database for Bank Service
    1. New class: `AccountDatabase.java`

```
/*
    This is a DB
```

```
        1 => 10
        2 => 20
        ...
        10 => 100
    */

    private static final Map<Integer, Integer> MAP = IntStream.rangeClosed(1, 10)
        .boxed()
        .collect(Collectors.toMap(
            Function.identity(),
            v -> v * 10)
        );

    public static int getBalance(int accountId) {
        return MAP.get(accountId);
    }

    public static Integer addBalance(int accountId, int amount) {
        return MAP.computeIfPresent(accountId, (k, v) -> v + amount);
    }

    public static Integer deductBalance(int accountId, int amount) {
        return MAP.computeIfPresent(accountId, (k, v) -> v - amount);
    }
```

1. `BankService`

   ```
   .setAmount(AccountDatabase.getBalance(accountNumber))
   ```

2. Restart the server

3. `BankClientTest.java`

BankService - A Simple DB Using Map - Note

1. Map is used as a DB
    1. Using gRPC, we are trying to replace REST
    2. Server to DB is TCP connection and will not change

[Resource] - Postman / BloomRPC Note

1. https://github.com/uw-labs/bloomrpc/releases - tool for gRPC
2. Postman also support gRPC

Unary - Postman - Demo

1. PostMan (another tool - Insomnia)
    1. Workspace
    2. gRPC Request BETA
        1. Enter server URL: localhost:6565
        2. Choose a way to load services and methods:
            1. Local file system: Click
                1. Select the proto file
                2. Next
                3. Use without importing
                    1. Message:
                        1. Click on: Generate Example Message
                            1. Give account number: 7
            2. URL

[DEPRECATED] - Unary - BloomRPC - Demo

1. A tool called bloomrpc like postman
    1. Install
    2. Launch
    3.
        - to load proto file
        1. Click on the proto file
            1. View Proto
            2. Server URL: localhost:6565
            3. Account no: 10

Unary - Node Client

1. Node client to access bank service

    1. `mkdir node-client`
    2. `cd node-client`
    3. `npm init -y`
    4. `npm install @grpc/proto-loader grpc` **(M)**
    5. `code .`

        1. new directory: `proto`

            1. `bank-service.proto`

               ```
               syntax = "proto";

               option java_multiple_files = true;
               option java_package = "com.vinsguru.models";

               message BalanceCheckRequest {
               ```

```
    int32 account_number = 1;
}

message Balance {
    int32 amount = 1;
}

service BankService {
    rpc
}
```

1. The options are ignored by javascript

2. bank-client.js

```
const grpc = require('grpc')
const protoloader = require('@grpc/proto-loader')

const packageDef = protoloader.loadSync('proto/bank-service.proto') // or else everything is anynchronous
const protoDesc = grpc.loadPackageDefinition(packageDef) // loading definitions using proto file

const client = new protoDesc.BankService('localhost:6565', grpc.credentials.createInsecure()) // request/response is in plaintext

client.getBalance({accountNumber: 4}, (err, balance) => {
    if (err) {
        console.error("Something bad happened")
    } else {
        console.log("Received : " + balance.amount)
    }
})
```

1. Terminal:

```
node bank-client.js
```

2. Other clients can be written using Ruby, Python, Go, ...

Section 4: gRPC - Server Streaming RPC

Server Streaming - Introduction

1. Client be sending one request to server and server will send multiple responses to the client (one or more)

Server Streaming - Service Definition

1. Withdraw request: Limit is $10 per transaction

2. bank-service.proto

```
message WithdrawRequest {
    int32 account_number = 1;
    int32 amount = 2;
}

message Money {
    int32 value = 1;
}

service BankService {
    // ...
    // server-side streaming
    rpc withdraw(WithdrawRequest) returns (stream Money);
}
```

1. Recompile

Server Streaming - Money Withdrawal Service Implementation

2. BankService.java

```
@Override
public void withdraw(WithdrawRequest request, StreamObserver<Money> responseObserver) {
    int accountNumer = request.getAccountNumer();
    int amount = request.getAmount(); // 10, 20, 30, 40, ...
    int balance = AccountDatabase.getBalance(accountNumer);

    for (int i = 0; i < (amount / 10); i++) {
        Money money = Money.newBuilder().setValue(10).build();
        responseObserver.onNext(money);
        AccountDatabase.deductBalance(accountNumber, 10);
        Thread.sleep(1000); // try-catch
    }

    responseObserver.onCompleted();
}
```

1. Server sends multiple responses
2. Start the server

Server Streaming - Postman Test

1. Open PostMan
    1. Reload and re-import the file
        1. withdraw - shows sign that we will get multiple responses if we send one request

2. Generate Example Message
3. Change the request to the following:

```
{
    "account_number": 7,
    "amount": 40
}
```

    1. We get 4 responses (every one second)

4. `getBalance`

[DEPRECATED] - Server Streaming - BloomRPC Test

1. BloomRPC
    1. Refresh
        1. Request is generated
        2. Change the request
        3. Run
            1. We get one response every one second
2. Instead of waiting for long time and send final response, the server sends responses in chunks more frequently

Server Streaming - Validations/Error Handling

1. Error handling:
    1. We have status codes in GRPC
2. `withdraw`
    1. `Status` class

```
FAILED_PRECONDITION
UNAUTHENTICATED
CANCELLED
ABORTED
ALREADY_EXISTS
DATA_LOSS
DEADLINE_EXCEEDED
INTERNAL
INVALID_ARGUMENT
```

    2. Code:

```
if (balance < amount) {
    Status status = Status.FAILED_PRECONDITION.withDescription("No enough money. You have only " + balance);
    responseObserver.onError(status.asRuntimeException());
    return;
}
```

        1. Restart the server

    3. BloomRPC

        1. Test with higher value

gRPC - Error Codes

1. Pre-defined error codes

Server Streaming - Blocking Client

1. `BankClientTest.java`

```
@Test
public void withdrawTest() {
    WithdrawRequest withdrawRequest = WithdrawRequest
        .newBuilder()
        .setAccountNumber(7)
        .setAmount(40)
        .build();
    this.blockingStub.withdraw(withdrawRequest)
            .forEachRemaining(money -> System.out.println("Received : " + money.getValue()));
}
```

    1. Use try-catch to catch the exception

Server Streaming - Async Client

1. Non-blocking stub:
2. `BankClientTest`

```
private BankServiceGrpc.BankServiceStub bankServiceStub;
...
@BeforeAll
public void setup() {
    ...
    this.bankServiceStub = BankServiceStub.newStub(managedChannel);
}
...
@Test
public void withdrawAsyncTest() {
    WithdrawRequest withdrawRequest = WithdrawRequest
        .newBuilder()
```

```
            .setAccountNumber(10)
            .setAmount(50)
            .build();
        this.bankServiceStub.withdraw(withdrawRequest, new MoneyStreamingResponse());
        Uninterruptibles.sleepUninterruptibly(3, TimeUnit.SECONDS);
    }
```

1. As and when money comes, `onNext()` will be executed
2. When no more money will be sent, `onCompleted()` will be executed

3. New Class: MoneyStreamingResponse.java

```
public class MoneyStreamingResponse implements StreamObserver<Money> {
    @Override
    public void onNext(Money money) {
        System.out.println("Received async : " + money.getValue());
    }

    @Override
    public void onError(Throwable throwable) {
        System.out.println(throwable.getMessage());
    }

    @Override
    public void onCompleted() {
        System.out.println("Server is done!!");
    }
}
```

Server Streaming - Count Down Latch

1. We usually don't wait using sleep statement
    1. `CountDownLatch` can be used instead (good for unit tests)

```
private CountDownLatch latch;

public MoneyStreamingResponse(CountDownLatch latch) {
    this.latch = latch;
}

@Override
public void onError(...) {
    ...
    latch.countDown();
}

@Override
public void onCompleted() {
    ...
    latch.countDown();
}

CountDownLatch latch = new CountDownLatch(1);
```

        1. `BankClientTest.java`

```
@Test
public void withdrawAsyncTest() throws InterruptedException {
    ...
    this.bankServiceStub.withdraw(withdrawRequest, new MoneyStreamingResponse(latch));
    latch.await();
}
```

Server Streaming vs Unary

1. `BankService.java`
    1. Add `responseObserver.onNext(balance)` multiple times
        1. Exception: CANCELLED HTTP/2
            1. Only one response is expected (signaturue looks same but only one response can be sent)

Section 5: gRPC - Client Streaming RPC

Client Streaming - Introduction

1. Client has a lot money and sends deposit requests. Once the deposits are done, client calls `onCompleted()`. Server finally responds by sending the balance.

Client Streaming - Service Definition

1. bank-service.proto

```
message DepositRequest {
    int32 account_number = 1;
    int32 amount = 2;
}

service BankService {
    ...
    rpc cashDeposit(stream DepositorRequest) returns (Balance);
}
```

Client Streaming - Bank - Deposit Request - Implementation - Part - 1

1. Re-compile the project

2. BankService.java

```
@Override
public StreamObserver<DepositRequest> cashDeposit(StreamObserver<Balance> responseObserver) {
    return super.cashDeposit(responseObserver);
}
```

    1. Request: We need to provide an implementation of `StreamObserver<DepositRequest>`

Client Streaming - Bank - Deposit Request - Implementation - Part - 2

1. New Class: `CashStreamingRequest.java`

```
public class CashStreamingRequest implements StreamObserver<DepositRequest> {
    private StreamObserver<Balance> balanceStreamObserver;
    private int accountBalance;

    public CashStreamingRequest(StreamObserver<Balance> balanceStreamObserver) {
        this.balanceStreamObserver = balanceStreamObserver;
    }

    @Override
    public void onNext(DepositRequest depositRequest) {
        int accountNumber = depositRequest.getAccountNumber();
        int account = depositRequest.getAmount();
        this.accountBalance = AccountDatabase.addBalance(accountNumber, amount);
    }

    @Override
    public void onError(Throwable throwable) {

    }

    @Override
    public void onCompleted() { // client invokes this
        Balance balance = Balance.newBuilder().setAmount(this.accountBalance).build();
        this.balanceStreamObserver.onNext(balance);
        this.balanceStreamObserver.onCompleted();
    }
}
```

    1. BankService.java

```
@Override
public StreamObserver<DepositRequest> cashDeposit(StreamObserver<Balance> responseObserver) {
    return new CashStreamingRequest(responseObserver);
}
```

Client Streaming - Client Response Observer Implementation

1. BankClientTest.java

```
@Test
public void cashStreamingRequest() throws InterruptedException {
    CountDownLatch latch = new CountDownLatch(1);
    StreamObserver<DepositRequest> streamObserver = this.bankServiceStub.cashDeposit(new BalanceStreamObserver(latch));
    for (int i = 0; i < 10; i++) {
        DepositRequest depositRequest = DepositRequest.newBuilder().setAccountNumber(8).setAmount(10).build();
        streamObserver.onNext(depositRequest);
    }
    streamObserver.onCompleted();
    latch.await();
}
```

    1. When client sends streaming request, we cannot do it in a blocking way
        1. It is asynchronous
    2. Server returns a handler using which we can send a stream of requests

2. BalanceStreamObserver.java

```
public class BalanceStreamObserver implements StreamObserver<Balance> {

    private CountDownLatch latch;

    public BalanceStreamObserver(CountDownLatch latch) {
        this.latch = latch;
    }

    @Override
    public void onNext(Balance balance) {
        System.out.println("Final Balance : " + balance.getAmount());
    }

    @Override
    public void onError(Throwable throwable) {
        this.latch.countDown();
    }

    @Override
    public void onCompleted() {
        System.out.println("Server is done!");
        this.latch.countDown();
    }
}
```

    1. It is automatically invoked when server sends response

Client Streaming - Postman Test

1. Re-import the file
2. Select `cashDeposit`
3. Generate Example Message

```
{
    "account_number": 3,
    "amount": 1
}
```

    1. Click: Invoke (opens the stream)
    2. Click: Send (multiple times)
    3. Click: End Stream

4. `getBalance`
    1. Send account number 3

Client Streaming - Stream Observer - How It Works

1. Client sends a stream of deposit requests
2. Server sends back the final balance as a response
3. The interfaces are shared between client and server

Section 6: gRPC - Bi-Directional Streaming RPC

BiDi - Introduction

1. Streaming requests and streaming responses
    1. The streams are independent
2. Example: Bank
    1. Client will be sending amounts to bank
        1. Transfer from account to to account
    2. Server will be respond the balances after each transfer
        1. It also sends success or failure

BiDi - Service Definition

1. transfer-service.proto

```
syntax = "proto3";

option java_multiple_files = true;
option java_pacakge = "com.vinsguru.models";

message TransferRequest {
    int32 from_account = 1;
    int32 to_account = 2;
    int32 amount = 3;
}

message Account {
    int32 account_number = 1;
    int32 amount = 2;
}

enum TransferStatus {
    FAILED = 0;
    SUCCESS = 1;
}

message TransferResponse {
    TransferStatus status = 1;
    repeated Account account = 2;
}

service TransferService {
    rpc transfer(stream TransferRequest) returns (stram TransferResponse);
}
```

BiDi - Transfer Service - Implementation

1. Compile
2. New class: TransferService.java

```
public class TransferService extends TransferServiceGrpc.TransferServiceImplBase {
    @Override
    public StreamObserver<TransferRequest> transfer(StreamObserver<TransferResponse> response) {

    }
}
```

3. TransferStreamingRequest.java

```
public class TransferStreamingRequest implements StreamObserver<TransferRequest> {
    private StreamObserver<TransferResponse> transferResponseStreamObserver;

    public TransferStreamRequest(StreamObserver<TransferResponse> transferResponseStreamObserver) {
        this.transferResponseStreamObserver = transferResponseStreamObserver;
    }

    @Override
```

```
        public void onNext(TransferRequest transferRequest) {
            int fromAccount = transferRequest.getFromAccount();
            int toAccount = transferRequest.getToAccount();
            int amount = transferRequest.getAmount();
            int balance = AccountDatabase.getBalance(fromAccount);
            TransferStatus status = TransferStatus.FAILED;

            if (balance >= amount && fromAccount != toAccount) {
                AccountDatabase.deductBalance(fromAccount, amount);
                AccountDatabase.addBalance(toAccount, amount);
                status = TransferStatus.SUCCESS;
            }
            Account fromAccountInfo = Account.newBuilder()
                .setAccountNumber(fromAccount)
                .setAmount(AccountDatabase
                .getBalance(fromAccount))
                .build();
            Account toAccountInfo = Account.newBuilder()
                .setAccountNumber(toAccount)
                .setAmount(AccountDatabase
                .getBalance(toAccount))
                .build();
            TransferResponse transferResponse = TransferResponse.newBuilder()
                .setStatus(status)
                .addAccount(fromAccountInfo)
                .addAccount(toAccountInfo)
                .build(); // building a response for each request
            this.transferResponseStreamObserver.onNext(transferResponse); // order is guaranteed so responses are sent in the same order we send in
        }

        @Override
        public void onError(Throwable throwable) {

        }

        @Override
        public void onCompleted() {
            AccountDatabase.printAccountDetails();
            this.transferResponseStreamObserver.onCompleted();
        }
    }
```

4. AccountDatabase.java

```
public static void printAccountDetails() {
    System.out.println(MAP);
}
```

BiDi - Client Side Stream Observer Implementation

1. TransferStreamingResponse.java

```
public class TransferStreamingResponse implements StreamObserver<TransferResponse> {
    private CountDownLatch latch;

    public TransferStreamingResponse(CountDownLatch latch) {
        this.latch = latch;
    }

    @Override
    public void onNext(TransferResponse transferResponse) {
        System.out.println("Status : " + transferResponse.getStatus());
        transferResponse.getAccountsList()
            .stream()
            .map(account -> account.getAccountNumber() + " : " + account.getAmount())
            .forEach(System.out::println);
        System.out.println("----------------------");
    }

    @Override
    public void onError(Throwable throwable) {
        this.latch.countDown();
    }

    @Override
    public void onCompleted() {
        System.out.println("All transfers are done");
        this.latch.countDown();
    }
}
```

BiDi - Client Side Stub Request And Response

1. New test: TransferClientTest.java

```
// copy setup info including @BeforeAll

private TransferServiceGrpc.TransferServiceStub stub;

@BeforeAll
public void setup() {
    ...
    this.stub = TransferServiceGrpc.newStub(managedClient);
}

@Test
public void transfer() throws InterruptedException {
    CountDownLatch latch = new CountDownLatch(1);
    TransferStreamingResponse response = new TransferStreamingResponse(latch);
    StreamObserver<TransferRequest> requestStreamObserver = this.stub.transfer(response);
```

```
            for (int i = 0; i < 100; i++) {
                TransferRequest.newBuilder()
                    .setFromAccount(ThreadLocalRandom.current().nextInt(1, 11))
                    .setToAccount(ThreadLocalRandom.current().nextInt(1, 11)
                    .setAmount(ThreadLocalRandom.current().nextInt(1, 21))
                    .build();
                requestStreamObserver.onNext(request);
            }
            requestStreamingObserver.onCompleted();
            latch.await();
        }
```

BiDi - Transfer Service Demo

1. GrpcServer.java

```
...
.addService(new TransferService())
...
```

    1. New server doesn't need to be created

2. TransferService.java

```
@Override
public ... transfer(...) {
    return new TransferStreamingRequest(responseObserver);
}
```

3. AccountDatabase.java

    1. All accounts must have $100 to start with

```
v -> 100
```

4. Start the server

    1. Client side: Transactions are shown
    2. Server side: Shows the final balance

BiDi - Single Response vs Streaming Response

1. RPC - a method invocation is considered as RPC
    1. Unary - each request is an RPC
    2. Streaming - It is just one RPC for each stream started
2. Response - Streaming vs Single
    1. **Streaming**
        1. Size - Potentially large / unknown
            1. Example: Pagination
                1. Example: Google - only first few pages are checked
        2. Receive side might take too much time to process
            1. File upload
    2. **Single**
        1. More efficient than streaming RPC
        2. Size is small

BiDi - How To Perform CRUD

1. gRPC CRUD:

```
service BookService {
    // GET
    rpc getBookById(BookRequest) returns (Book);

    // POST
    rpc saveBook(Book) returns (Book);

    // PUT
    rpc updateBook(Book) returns (Book);

    // DELETE
    rpc deleteBook(BookRequest) returns (Void);
}

message Void{}
```

    1. Similar to REST CRUD

Section 7: Snakes & Ladders - Game / Assignment

Game - Introduction

1. Board game

[Optional]-[Resource]-Rules

1. https://www.ymimports.com/pages/how-to-play-snakes-and-ladders

Game - Hints

Game - Service Definition

Game - Die Streaming Observer Implementation

Game - Applying Snakes & Ladders Rules

Game - Players State Streaming Observer Implementation

Small Mistake - Correction

Game - Client Server Finale

Section 8: Channel & Load Balancing

Load Balancing - Introduction

1. HTTP 2.0 increases network efficiency by providing multi-plexing
    1. We don't have to wait for a response before sending another request
        1. We can send multiple requests in parallel on one HTTP 2.0 connection
2. gRPC Channel

    1. Example:

    ```
    ManagedChannelBuilder
        .forAddress("localhost", 6565)
        .usePlaintext()
        .build();
    ```

        1. It constructs a channel
            1. To establish a connection between a client and a server
            2. **Channel**: It is an abstraction over an HTTP 2.0 connection between client and server (it represents a connection)
                1. It has logic to keep the connection healthy, alive, and make use of them properly
                2. It is a persistent connection
                    1. 30 minutes by default (but configurable)
            3. A connection is created if there is no active connection, once it is created and available, it is used for multiple concurrent requests between client and server
            4. A connection can be closed
                1. Idle connection can be closed by the server
                    1. Wastes resources
                2. When a newer version of the app is deployed
                3. When our app crashes
                    1. Out of memory say
            5. It is okay if connection is closed
                1. Channel will automatically make a new connection if it does not find any active connection
            6. Channel is thread-safe
                1. We can share it with multiple stubs
                2. We can make multiple parallel rpc requests
            7. Channel creation is expensive
                1. Hence, we will not be instantiating a channel for each and every server call
                2. **One channel is created for one backend at server startup**
                3. We can shutdown the channel when we no longer need the channel
            8. If there are multiple service definitions for the backend, we will have multiple stubs for the same backend
                1. We can have one managed channel and we can share the channel for all the stubs

    2. Channel is an abstraction oer a connection and represents the connection

    3. Connection is persistent

    4. Connection creation is lazy & established during the first RPC

    5. 1 channel / connection is enough for client/server communication even for concurrent requests

        1. We can also construct more channels - but not really required
        2. Channel creation is an expensive process

    6. Close when the server shuts down

    7. Thread safe!

    8. Can be shared with multiple stubs for the server

Load Balancing - Lazy Connection Demo

1. BankClientTest.java

```
@BeforeAll
public void steup() {
    // change port to 7200
    ...
    System.out.println("Channel is created");
    ...
    System.out.println("Stubs are created");
}

@Test
public void balanceTest() {
    Thread.sleep(5000);
    ...
}
```

    1. When channel is created, not HTTP 2.0 connection is created

Load Balancing - Server Side Load Balancing

1. Example: order-service wants to make request to payment-service
    1. In HTTP 2.0 it is a persistent connection
        1. How does the load balancing work?
            1. There are two types of load balancing
                1. Server-side load balancing
                2. Client-side load balancing
2. Server-side load balancing
    1. A proxy is used (ALB, ...)
        1. It accepts request and forwards to one of the instances on the backend side
        2. The client doesn't need to know about the instances on the backend side
        3. A persistent connection can be established between the client and the proxy service
        4. The proxy service will forward
            1. Nginx will be used to make persistent connection

Load Balancing - Nginx Install

1. Windows:

```
cd c:\
unzip nginx-1.19.3.zip
cd nginx-1.19.3
start nginx
```

    1. `conf` - config file to make changes

2. Mac:

```
// install
brew install nginx

// conf
/usr/local/etc/nginx/nginx.conf
```

Load Balancing - Project Setup

1. Remove all the changes made previously
2. New package: `client/rpcTypes`
    1. Move all the client code under this
3. New package: `server/rpcTypes`
    1. Move all the client code under this
4. New package: `server/loadBlancing`
    1. Copy `BankService`
    2. Copy `GrpcServer`
5. New package: `client/loadBalancing`

Load Balancing - Running Multiple gRPC Servers

1. Copy `GrpcServer` to `GrpcServer1` and `GrpcServer2`

    1. Change ports to `6565` and `7575`

2. `BankService`

```
@Override
public void getBalance(...) {
    //...
    System.out.println("Received the request for " + accountNumber);
    //...
}
```

3. Start `GrpcServer1` and `GrpcServer2`

[Resource] - Nginx Conf

1. Install Nginx: https://www.nginx.com/resources/wiki/start/topics/tutorials/install/

2. Docker:

```
version: "3"
services:
    nginx:
        image: nginx:1.15-alpine
        volumes:
            - ./conf:/etc/nginx/conf.d
        ports:
            - 8585:8585
```

    1. Update IPs as per the machine (we can use `localhost` if not using Docker (`./conf/default.conf`)

```
upstream bankservers {
    server 172.17.0.1:6565;
    server 172.17.0.1:7575;
}

server {
    listen 8585 http2;

    location / {
        grpc_pass grpc://bankservers;
    }
}
```

Load Balancing - Nginx Conf

1. Nginx listens on port 8585
2. We need to explicitly state that we need `http2`
3. `grpc_pass` - forwards to backend servers (`bankservers`)

Load Balancing - Nginx Load Balancing Demo - Part - 1

1. `loadBalancing/NginxTestClient.java`

```
// Copy setup from previous test file

ManagedChannel managedChannel = ManagedChannelBuilder.forAddress("localhost", 8585)
    .usePlaintext()
    .build();

@Test
public void balanceTest() {
    for (int i = 0; i < 11; i++) {
        BalanceCheckRequest balanceCheckRequest = BalanceCheckRequest.newBuilder()
            .setAccountNumber()
            .build();
        Balance balance = this.blockingStub.getBalance(balanceCheckRequest);
        System.out.println("Received : " + balance.getAmount());
    }
}
```

    1. The servers got requests in round-robbin fashion

Load Balancing - Nginx Load Balancing Demo - Part - 2

1. Send 100 requests

```
.setAccountNumber(ThreadLocalRandom.current().nextInt(1, 111)
    .setAccountNumber()
    .build();
try {
    Thread.sleep(500);
} catch (InterruptedException e) {
    e.printStackTrace();
}
```

2. Nginx forwards request to one of the healthy backend servers

Load Balancing - Client Streaming Request

1. BankService.java

Load Balancing - SubChannels

Load Balancing - A Simple Service Registry Implementation

Load Balancing - Name Resolver

[UPDATE] - grpc-java library version 1.41

Load Balancing - Name Resolver Provider

Load Balancing - Client Side Load Balancing Demo

Load Balancing - Summary

Section 9: Deadline

Deadline - Introduction

1. Deadline - It is a timeout for an RPC to complete!
    1. How long will client wait before it will not accept response
        1. Makes it resilient
            1. We don't want to wait indefinitely (best practice)

Deadline - Project Setup

1. New package: `server/deadline/BankService.java` and `server/deadline/GrpcServer.java`
2. Rename them to `BankService.java` to `DeadlineService`
3. New package: `test/java/com/vinsguru/client/deadline/DeadlineClientTest.java`

Deadline - Unary RPC - Demo

1. DeadlineService.java

```
@Override
public void getBalance(...) {
    ...
    Uninterruptibles.sleepUninterruptibly(3, TimeUnit.SECONDS);
    responseObserver.onNext(balance);
    responseObserver.onCompleted();
}
```

2. GrpcServer.java
    1. Start the server
3. DeadlineClientTest.java

```
@Test
public void balanceTest() {
    ...
}
```

    1. Run it
        1. It takes around 3 seconds
4. With deadline:
    1. DeadlineClientTest.java

```
@Test
public void balanceTest() {
    //...
    Balance balance = this.blockingStub
            .withDeadline(Deadline.after(2, TimeUnit.SECONDS))
            .getBalance(balanceCheckRequest);
    // ...
}
```

        1. Throws exception
        2. Use a try/catch block to handle the exception
            1. Go with default value if an exception is thrown

Deadline - Server Streaming RPC Issue

1. Three seconds delay for each $ 10

```
for (int i = 0; i < (amount/10); i++) {
    // ...
    Uninterruptibles.sleepUninterruptibly(3, TimeUnit.SECONDS);
    // ...
}
```

2. DeadlineClientTest.java

```
try {
    this.blockingStub
            .withDeadline(Deadline.after(3, TimeUnit.SECONDS))
            ...
} catch (StatusRuntimeException e) {
    // ...
}
```

    1. Client has left but server still sends responses (problem)

Deadline - Server Streaming RPC - Issue Fix

1. Context - namespace or current rpc to store and retrieve information
    1. It provides info to check if the current rpc is alive or not
2. DeadlineService.java

```
Uninterruptibes.sleepUninterruptibly(3, TimeUnit.SECONDS);
if (!Context.current().isCancelled()) {
    responseObserver.onNext(money);
    System.out.println("Delivered $10");
    AccountDatabase.deductBalance(accountNumber, 10);
} else {
    break;
}
// ...
System.out.println("Cancelled");
// ...
```

Section 10: Interceptors

Interceptor - Introduction

1. Interceptor:
    1. We can intercept RPCs both at client and server side
        1. To handle Cross-cutting concerns
            1. Logging
            2. Monitoring
            3. Rate-limiting
            4. Deadline
            5. Authentication
            6. ...
    2. They provide interfaces for us to intercept any rpc calls at client and server side
        1. Keeps business rules clean from cross-cutting concerns

Interceptor - Global Deadline Implementation

1. Configuring Deadline in one place

1. `com.vinsguru.client.deadline.DeadlineInterceptor.java`

```
public class DeadlineInterceptor implements ClientInterceptor {
    @Override
    public <ReqT, RespT> ClientCall<ReqT, RespT> interceptCall(MethodDescriptor<ReqT, RespT> methodDescriptor, CallOptions callOptions, Ch

    }
}
```

    1. `methodDescriptor` - what is the method we are trying to invoke
    2. `callOptions` - the options we are setting in our call as part of the stub
    3. `channel` - used in the call
    4. Simply invoke the remote call:

```
return channel.newCall(methodDescriptor, callOptions);
```

    5. Setting deadline:

```
retunr channel.newCall(methodDescriptor, callOptions.withDeadline(Deadline.after(4, TimeUnit.SECONDS));
```

    6. DeadlineClientTest.java

```
ManagedChannel managedChannel = ManagedChannelBuilder.forAddress("localhost", 6565)
    .intercept(new DeadlineInterceptor())
    .usePlainText()
    .build();
```

        1. Remove `withDeadline` from the code

Interceptor - Global Deadline - Demo

1. Run the client test
2. How to override global timeout?
3. DeadlineInterceptor.java

```
Deadline deadline = callOptions.getDeadline(); // null if not set

if (Objects.isNull(deadline)) {
    callOptions = callOptions.withDeadline(Deadline.after(4, TimeUnit.SECONDS));
}
return channel.newCall(methodDescriptor, callOptions);
```

4. DeadlineClientTest.java

```
this.blockingStub
        .withDeadline(Deadline.after(2, TimeUnit.SECONDS)
        ...
```

Interceptor - Key/Value Pairs - Introduction

1. When we invoke a method using a stub, request is initiated and interceptor can see the request
    1. We can log
    2. We can attach a token
    3. We can set deadline
2. Then the request is forwarded
3. Once the request reaches the server, the server interceptor intercepts the request (optional)
    1. It can rate limit
        1. Based on quota
    2. ...
4. Token:
    1. Interceptor on client side generates a token
    2. Interceptor on server side can validate the token
5. We need more tools for that
    1. `CallOptions`: To pass information from client stub to interceptor
        1. To set deadline say
            1. Interceptor sees that
                1. If the deadline is set, the interceptor uses that
                2. If the deadline is not set, the interceptor sets it
    2. `Metadata`: If client needs to pass some info to server, we use metadata
        1. It is Header in HTTP 2.0
        2. Key/Value pair
            1. Session token
            2. ...
    3. `Context`: Used to pass info extracted in the server side interceptor to business layer
        1. It is a namespace used to store and retrieve info specific to an RPC call on server side

Interceptor - Attaching Client Service Token Via Metadata

1. Example: Only authorized clients need to send request to server
    1. Client will always attach token in header
    2. User request token can also be attached
2. `metadata/MetadataClientTest.java`

```
@BeforeAll
public ...() {
```

```
        ManagedChannel managedChannel = ...
            .intercept(MetadataUtils.newAttachHeadersInterceptor(ClientConstants.getClientToken()))
            ...
    }
```

1. Grpc provides an interceptor for us and it will attach the metadata we are giving in request

3. `metadata/ClientConstants.java`

```
public class ClientConstants {

    private static final Metadata METADATA = new Metadata();

    static {
        METADATA.put(
            Metadata.Key.of("client-token", Metadata.ASCII_STRING_MARSHALLER),
            "bank-client-secret"
        );
    }

    public static Metadata getClientToken() {
        return METADATA;
    }
}
```

1. `Metadata`: Key-value pairs
    1. Headers are strings in HTTP 1.1
    2. Headers support different data types in HTTP 2.0
        1. Byte array
        2. String
        3. Protobuf object
    3. Used for deserialization

Interceptor - Client Service Token Validation - Implementation

1. Server-side interceptor to extract the information
2. `server/metadata`
    1. Copy paste: `GrpcServer.java` and `DeadlineService.java`
        1. Rename `DeadlineService.java` to `MetadataService.java`
    2. New class: `AuthInterceptor.java`

```
public class AuthInterceptor implements ServerInterceptor {
    @Override
    public <ReqT, RespT> ServerCall.Listener<ReqT> interceptCall(ServerCall<ReqT, RespT> serverCall, Metadata metadata, ServerCallHandler<
        String clientToken = metadata.get(ServerConstants.TOKEN); // string marshaller is used, any other object can be used
        if (this.validate(clientToken) {
            return serverCallHandler.startCall(serverCall, metadata);
        } else {
            Status status = Status.UNAUTHENTICATED.withDescription("invalid token/ expired token");
            serverCall.close(status, metadata);
        }
        return new ServerCall.Listener<ReqT>() {}; // null must not be sent
    }

    private boolean validate(String token) {
        return Objects.nonNull(token) && token.equals("bank-client-secret");
    }
}
```

    3. `metadata/ServerConstants.java`

```
public class ServerConstants {
    public static final Metadata.Key<String> TOKEN = Metadata.Key.of("client-token", Metadata.ASCII_STRING_MARSHALLER);
}
```

        1. `TOKEN` - used to extract information

Interceptor - Client Service Token Validation - Demo

1. GrpcServer.java

```
Server server = ServerBuilder.forPort(6565)
                    .intercept(new AuthInterceptor())
                    .addService(new MetadataService())
                    .build();

server.start();
server.awaitTermination();
```

2. MetadataClientTest.java
    1. Run it
3. ClientConstants.java

```
METADATA.put(
    ...
    "bank-client-secre"
);
```

    1. Run MetadataClientTest.java by printing stack-trace

```
    catch (StatusRuntimeException e) {
        e.printStackTrace();
    }
```

Interceptor - Attaching User Session Token - Introduction

1. Client sends info vie JWT

Interceptor - Attaching User Session Token - Implementation

1. Each request will be having a token specific to the user
2. Grpc provides an abstract class that we need to extend
    1. `metadata/UserSessionToken.java`

```
public class UserSessionToken extends CallCredentials {

    public static final Metadata.Key<String> USER_TOKEN = Metadata.Key.of("user-token", Metadata.ASCII_STRING_MARSHALLER);
    private String jwt;

    public UserSessionToken(String jwt) {
        this.jwt = jwt;
    }

    @Override
    public void applyRequestMetadata(RequestInfo requestInfo, Executor executor, MetadataApplier ) {
        executor.execute(() -> {
            Metadata metadata = new Metadata();
            metadata.put(ClientConstants.USER_TOKEN, this.jwt);
            metadataApplier.apply(metadata);
            // metadataApplier.fail(
        });
        // Executor - if the class is used to generate a token, it should not be blocking and it should be super quick
    }

    @Override
    public void thisUsesUnstableApi() { // just a note
        // may change in future
    }
}
```

    2. `metadata/ClientConstants.java`

```
public static final Metadata.Key<String> USER_TOKEN = Metadata.Key.of("user-token", Metadata.ASCII_STRING_MARSHALLER);
```

Interceptor - User Session Token Validation - Implementation

1. Server side:
2. ServerConstants.java

```
public static final Metadata.Key<String> USER_TOKEN = Metadata.Key.of("user-token", Metadata.ASCII_STRING_MARSHALLER);
```

    1. Two sets of keys
        1. Server keys
        2. Client keys
3. AuthInterceptor.java

```
String clientToken = metadata.get(ServerConstants.USER_TOKEN);
// ...

private boolean validate(String token) {
    return Objects.nonNull(token) && token.equals("user-secret-3");
}
```

    1. We can have a separate interctor as well
4. MetadataService.java

```
// Remove hardcoded sleep
```

5. Start the server

Interceptor - User Session Token Validation - Demo

1. MetadataClientTest.java
    1. We need to attach user-token for each and every method call

```
@Test
public void balanceTest() {
    ...
    for (int i = 0; i < 20; i++) {
        try {
            int random = ThreadLocalRandom.current().nextInt(1, 4);
            System.out.println("Random : " + random);
            Balance balance = this.blockingStub
                        .withCallCredentials(new UserSessionToken("user-secret-" + random))
                        .getBalance(balanceCheckRequest);
            System.out.println("Received : " + balance.getAmount());
        } catch (StatusRuntimeException e) {
            e.printStackTrace();
        }
    }
}
```

        1. Every 3 passed but every other number failed

Interceptor - Context - Introduction

1. Example: Primium users view balance for free but non-premium users can view the balance for a fee (say)
    1. We can pass user-role as part of the protobuf message (as part of the payload)
        1. Easy to implement
    2. We can use Metadata & Context to send user-role
        1. Metadata carries user-role
        2. Authentication interceptor receives the information
            1. User role is extracted here
    3. Context carries the user-role to the business rule
        1. Context - namespace to store and retrieve information specific to an RPC call

Interceptor - User Role - Auth Interceptor Implementation

1. Extract information at interceptor and pass it to service layer (one way to implement it)

    1. Another way: We can get the user id and make a database call to get the user-role information and pass it to the service layer

2. AuthInterceptor.java

```
/*
    user-secret-3 prime
    user-secret-3 regular
*/
if (this.validate(clientToken)) {
    UserRole userRole = this.extractUserRole(clientToken);
    Context context = Context.current().withValue(
        ServerConstants.CTX_USER_ROLE,
        userRole
    );
    return Contexts.interceptCall(context, serverCall, metadata, serverCallHandler);
}
// ...
private boolean validate(String token) {
    return Objects.nonNull(token) &&
        (token.startsWith("user-secret-3") || token.startsWith("user-secret-2"));
}

private UserRole extractUserRole(String jwt) {
    return jwt.endsWith("prime") ? UserRole.PRIME : UserRole.STANDARD;
}
```

3. UserRole.java

```
public enum UserRole {
    PRIME,
    STANDARD
}
```

4. ServerConstants.java

```
public static final Context.Key<UserRole> CTX_USER_ROLE = Context.key("user-role");
public static final Context.Key<UserRole> CTX_USER_ROLE1 = Context.key("user-role");
```

Interceptor - Accessing User Role Via Context

1. MetadataService.java

```
@Override
public void getBalance(...) {
    //...
    UserRole userRole = ServerConstants.CTX_USER_ROLE.get();
    UserRole userRole1 = ServerConstants.CTX_USER_ROLE1.get();
    amount = UserRole.PRIME.equals(userRole) ? amount : (amount - 15);

    System.out.println(userRole + " : " + userRole1);

    Balance balance = Balance.newBuilder()
        .setAmount(amount)
        .build();
    //...
}
```

    1. Start the server

Interceptor - Context Demo

1. MetadataClient.java

```
try {
    //...
    Balance balance = this.blockingStub
        .withCallCredentials(new userSessionToken("user-secret-" + random + ":prime"))
        // ...
```

    1. Send the request

    2. Standard user:

```
try {
//...
Balance balance = this.blockingStub
    .withCallCredentials(new userSessionToken("user-secret-" + random + ":standard"))
    // ...
```

Section 11: Error Handling Via Metadata

Error Handling - Introduction

1. gRPC - Error Handling
    1. Error Channel - used to respond to client request
        1. Status codes
            1. Used to throw exceptions
        2. Metadata
    2. Data Channel
        1. OneOf
2. Not having enough money can be treated as outcome of request instead of a system error
    1. We can also use the Data Channel instead of the Error Channel
        1. Instead of throwing exceptions
            1. We need to close the data channel excplicitly but not the error channel
                1. Error channel is closed implicitly
3. There could be many reasons that a request can fail
    1. We need custom codes because status codes are limited
4. Metadata
    1. Server can send info back to client using metadata
    2. It is not a string
        1. We can send binary or any reasonably sized object

Error Handling - Proto For Custom Error Message

1. bank-service.proto

```
enum ErroMessage {
    ONLY_TEN_MULTIPLES = 0;
    INSUFFICIENT_BALANCE = 1;
}

message WithDrawalError {
    ErrorMessage error_message = 1;
    int32 amount = 2;
}
```

    1. Compile it

Error Handling - Service Layer Charges

1. MetadataService.java

```
@Override
public void withdraw(...) {
    //...

    if (amount < 10 || (amount % 10 != 0)) {
        Metadata metadata = new Metadata();

        Metadata.Key<WithdrawalError> errorKey = ProtoUtils.keyForProto(WithdrawalError.getDefaultInstance());

        WithdrawalError withdrawalError = WithdrawalError.newBuilder().setAmount(balance).setErrorMessage(ErrorMessage.ONLY_TEN_MULTIPLES).buil

        metadata.put(errorKey, withdrawalError);

        responseObserver.onError(Status.FAILED_PRECONDITION.asRuntimeException(metadata));

        return;
    }

    if (balance < amount) {
        Metadata metadata = new Metadata();

        Metadata.Key<WithdrawalError> errorKey = ProtoUtils.keyForProto(WithdrawalError.getDefaultInstance());

        WithdrawalError withdrawalError = WithdrawalError.newBuilder().setAmount(balance).setErrorMessage(ErrorMessage.INSUFFICIENT_BALANCE).bu

        metadata.put(errorKey, withdrawalError);

        responseObserver.onError(Status.FAILED_PRECONDITION.asRuntimeException(metadata));

        return;
    }

    //...
    for (int i = 0; i < (amount/10); i++) {
        Money ...;
        responseObserver.onNext(money);
        AccountDatabase.deductBalance(accountNumber, 10);
    }
}
```

    1. Remove Authentication

```
  // .interce(new AuthInterceptor())
```

Error Handling - Custom Error Via Metadata

1. MoneyStreamingResponse.java

```
@Override
public void onError(Throwable throwable) {
    Status.fromThrowable(throwable); // to convert throwable into Status object
    Metadata metadata = Status.trailersFromThrowable(throwable); // to get metadata from throwable
    metadata.get(ClientConstants.WITHDRAWAL_ERROR_KEY);
    System.out.println(
        withdrawalError.getAmount() + ": " + withdrawalError.getErrorMessage()
```

```
        );
    }
```

2. ClientConstants.java

```
public static final Metadata.Key<WithdrawalError> WITHDRAWAL_ERROR_KEY = ProtoUtils.keyForProto(WithdrawalError.getDefaultInstance());
```

3. Run the server

4. Test:

```
@Test
public void withdrawAsyncTest() throws InterruptedException {
    // ...
    ...setAmount(37)...
}
```

    1. Output:

```
50:ONLY_TEN_MULTIPLES
```

    2. Insufficient balance:

```
...setAmount(60)...
```

        1. Output:

```
10:INSUFFICIENT_BALANCE
```

Error Handling - OneOf

1. Example

Section 12: SSL/TLS

SSL/TLS - Introduction

1. Secure connections with gRPC
2. Consider a bank application
    1. Client needs to confirm that it is talking to the actual bank application
        1. CA offers a certificate to the server
        2. Server sends the certificate to the client
            1. SSL/TLS handshake is used
        3. OS comes with CA certificates using which we can verify if we have an approved certificate
3. CA:
    1. We can become one
        1. We can be internal CA in an org
4. Steps:
    1. We become CA
    2. We generate certificate
    3. Client is updated with CA

[Optional-Resource] - SSL/TLS

1. Download: https://vins-udemy.s3.amazonaws.com/grpc-java/ssl-tls.zip

2. Become a CS:

    1. Generate a private key first for CA (des3 is encryption standard)

```
openssl genrsa -des3 -out ca.key.pem 2048
```

    2. Generate a ca certificate (This certificate is used to sign the server certificate and client will use this later)

```
openssl req -x509 -new -nodes -key ca.key.pem -sha256 -days 365 -out ca.cert.pem
```

3. Server:

    1. Generate a private key for the server

```
openssl genrsa -out localhost.key 2048
```

    2. As service owner, we generate a request for seding to CA

```
openssl req -new -key localhost.key -out localhost.csr
```

4. CA signs our request:

    1. CA signs our request which we need to keep safe

```
openssl x509 -req -in localhost.csr -CA ca.cert.pem -CAkey ca.key.pem -CAcreateserial -out localhost.crt -days 365
```

5. gRPC specific step:

    1. grpc expects the localhost key in PKCS8 standard

```
openssl pkcs8 -topk8 -nocrypt -in localhost.key -out localhost.pem
```

6. Open SSL: https://jamielinux.com/docs/openssl-certificate-authority/index.html

SSL/TLS - Becoming CA

1. New directories: `ssl-tls`
2. Becoming a CA:
    1. Generate private key - give some password: `openssl genrsa -des3 -out ca.key.pem 2048`
    2. Generate certificate (with public key): `openssl req -x509 -new -nodes -key ca.key.pem -sha256 -days 365 -out ca.cert.pem`
        1. Use the password used before
        2. Don't give any details

SSL/TLS - Generating Certificates

1. Generating private key for server: `openssl genrsa -out localhost.key 2048`
2. Generate certificate signing request: `openssl req -new -key localhost.key -out localhost.csr`
    1. Don't give any details
    2. Common Name: localhost
    3. Everything else is optional
3. Sign the certificate: `openssl x509 -req -in localhost.csr -CA ca.cert.pem -CAkey ca.key.pem -CAcreateserial -out localhost.crt -day 365`
    1. Give password for CA private key
4. Converting key file to PKCS8 format: `openssl pkcs8 -topk8 -nocrypt -in localhost.key -out localhost.pem`

SSL/TLS - Server Side Changes

1. `ssl/BankService`, `ssl/GrpcServer`

2. BankService.java

```
SslContext sslContext = GrpcSslContexts.configure(
    SslContextBuilder.forServer(
        new File("<path>/localhost.crt"),
        new File("<path>/localhost.pem")
    )
).build();

Server server = NettyServerBuilder.forPort(6565)
                    .sslContext(sslContext)
                    .addService(new BankService())
                    .build();

server.start();
server.awaitTermination();
```

SSL/TLS - Demo

1. BankClientTest.java

```
@BeforeAll
public void setup() {
    SslContext sslContext = GrpcSslContext.forClient()
        .trustManager(new File("<path>/ca.cert.pem"))
        .build();

    ManagedChannel managedChannel = NettyChannelBuilder.forAddress("localhost", 6565)
        .sslContext(sslContext)
        .build();
}
```

Section 13: Miscellaneous

Misc - Executors

1. gRPC - Executor
    1. `CacheThreadPool` - default
    2. `directExecutor` - Better performance for non-blocking app
    3. `FixedThreadPool` - Perferred
2. Netty Server:
    1. There are two thread groups
        1. Boss Thread Group
            1. Responsibility:
                1. Listen on socket
                2. Accept incoming RPC
                3. Do SSL/TLS handshake
        2. Worker Thread Group
            1. They are also called event loops
                1. Usually one or two times the number of cores (configurable)
            2. Responsibility:
                1. Read requests and assign to:
                    1. CachedThreadPool - by default OR
                        1. If there is an existing thread, use it
                        2. If there are no existing threads, instantiate a new thread
                        3. The threads in this pool do the actual service
                    2. direct Executor
                        1. Worker event loops will accept the request, process the request and respond
                            1. Better performance - only for non-blocking applications
                                1. If event loop thread will get blocked if there is an IO call (database call, say)
                                2. The limited number of worker threads might get blocked and might make the performance worse
                    3. `FixedThreadPool` - Community suggests this

                        1. Dedicated threads to process the requests

                        2. How to configure?

```
ServerBuilder.forPort(6565)
    .directExecutor()
    .addService(new GameService())
    .build();

ServerBuilder.forPort(6565)
    .executor(Executors.newFixedThreadPool(20))
    .addService(new GameService())
    .build();
```

1. Measure performance and go with the option that works

Misc - Accessing gRPC via Web

1. gRPC - Web
    1. Browsers support HTTP/2 (for downloading files)
    2. However, Browser doesn't have an API to get the notification (in client-side JS) if server pushes data
        1. Solution: A new microservice with HTTP/1 API (REST API, GraphQL)
            1. Backend-for front-end service
        2. Solution: gRPC Web (a proxy) - Envoy based
            1. Is between gRPC backend and the browser

Section 14: Spring Boot Integration

Spring Boot Integration - Introduction

1. There is no official Spring Boot starter
    1. There are community ones
2. Steps - Server:
    1. pom.xml

```
<dependency>
    <groupId>net.devh</groupId>
    <artificatId>grpc-server-spring-boot-starter</artifactId>
    <version>2.9.0.RELEASE</version>
</dependency>
```

    2. application.properties

```
grpc.server.port=6565
```

    3. Service:

```
@GrpcService
public class CalculatorService extends CalculatorServiceGrpc.CalculatorServiceImplBase {
    @Override
    public void getAllDoubles(Input request, StreamObserver<Output> responseObserver) {
        int index = request.getNumber();
        IntStream.rangeClosed(1, index)
            .map(i -> i * 2) // add Thread.sleep to simulate time consuming operation
            .mapToObj(i -> Output.newBuilder().setResult(i).build())
            .forEach(responseObserver::onNext);
        responseObserver.onCompleted();
    }
}
```

3. Steps - Client:
    1. Dependency

```
<depencency>
    <groupId>net.devh</groupId>
    <artifactId>grpc-client-spring-boot-starter</artifactId>
    <version>2.9.0-RELEASE</version>
</dependency>
```

    2. application.yaml

```
grpc:
    client:
        calculator-service: # any name - a channel is atomatically created using the name - this name is used for creation of stub
            address: static://localhost:6565 # where service is running
            negotiationType: plaintext # remove if SSL/TLS
```

    3. Stub:

```
@GrpcClient("calculator-service")
private CalculatorServiceGrpc.CalculatorServiceBlockingStub blockingStub;
```

[2023] - Updated Maven Dependencies

1. The videos are old.
    1. Code with latest Spring ang gRPC

Spring Boot Integration - Application Walk-Through

1. Architecture:

```
Browser
   |
   v
```

```
aggregator service
    |
    +-> user-service -> DB (H2)
    |
    +-> movie-service -> DB
```

1. When user logs-in, based on the user-id, aggregator service asks for user's favorite genre
2. Aggregator uses the genre info and fetches all movies
3. Aggregator sends the movies to the user to try

Spring Boot Integration - Spring Initializer

1. Spring initializer
    1. Group: com.vinsguru
    2. Artifact: grpc-fix
    3. Name: grpc-fix
    4. Description: Demo project for Spring Boot
    5. Package name: com.vinsguru.grpc-fix
    6. Pacaging: jar
    7. Dependencies:
        1. Spring Web
        2. Lombok
        3. H2 Database
        4. Spring Data JPA

Spring Boot Integration - Multi Module Maven Project

1. Multiple Maven Modules
    1. Each service is a maven module
2. New Modules
    1. Name: proto (only proto files)
        1. pom.xml
            1. Paste all the dependencies of proto project
        2. src/main/proto
    2. Name: user-service
        1. pom.xml

```
<dependencies>
    <dependency>
        <groupId>net.devh</groupId>
        <artifactId>grpc-server-spring-boot-starter</artifactId>
        <version>2.9.0.RELEASE</version>
    </dependency>
    <!-- grpc-server-spring-boot-starter -->
    <!-- spring-boot-starter-data-jpa -->
    <!-- h2 -->
    <!-- lombok -->
    <dependency>
        <groupId>com.vinsguru</groupId>
        <artifactId>proto</artifactId>
        <version>0.0.1-SNAPSHOT</version>
    </dependency>
</dependencies>
```

    3. Name: movie-service
        1. pom.xml

```
<dependencies>
    <dependency>
        <groupId>net.devh</groupId>
        <artifactId>grpc-server-spring-boot-starter</artifactId>
        <version>2.9.0.RELEASE</version>
    </dependency>
    <!-- grpc-server-spring-boot-starter -->
    <!-- spring-boot-starter-data-jpa -->
    <!-- h2 -->
    <!-- lombok -->
    <dependency>
        <groupId>com.vinsguru</groupId>
        <artifactId>proto</artifactId>
        <version>0.0.1-SNAPSHOT</version>
    </dependency>
</dependencies>
```

    4. Name: aggregator-service
        1. pom.xml

```
<dependencies>
    <!-- spring-boot-starter-web -->
    <!-- lombok -->
    <!-- grpc-client-spring-boot-starter -->
    <dependency>
        <groupId>net.devh</groupId>
        <artifactId>grpc-client-spring-boot-starter</artifactId>
        <version>2.9.0.RELEASE</version>
    </dependency>
    <dependency>
        <groupId>com.vinsguru</groupId>
        <artifactId>proto</artifactId>
        <version>0.0.1-SNAPSHOT</version>
```

```
        </dependency>
    </dependencies>
```

5. Remove the 4 dependencies from the main pom.xml
    1. First module is proto

Spring Boot Integration - Service Definition

1. proto/src/main/proto
    1. File > Project Structure > Make it source
    2. movie-service.proto

```
syntax = "proto3";

import "common/common.proto";

option java_multiple_files = true;
option java_pacakge = "com.vinsguru.grpcflix.movie";

message MovieDto {
    string title = 1;
    int32 year = 2;
    double rating = 3;
}

message MovieSearchRequest {
    common.Genre genre = 1;
}

message MovieSearchResponse {
    repeated MovieDto movie = 1;
}

service MovieService {
    rpc getMovie(MovieSearchRequest) returns (MovieSearchResponse);
}
```

    3. user-service.proto

```
syntax = "proto3";

import "common/common.proto";

option java_multiple_files = true;
option java_package = "com.vinsguru.grpcflix.user";

message UserSearchRequest {
    string login_id = 1;
}

message UserResponse {
    string login_id = 1;
    string name = 2;
    common.Genre genre = 3;
}

message UserGenreUpdateRequest {
    string login_id = 1;
    common.Genre genre = 2;
}

service UserService {
    rpc getUserGenre(UserSearchRequest) returns (UserResponse);
    rpc updateUserGenre(UserGenreUpdateRequest) returns (UserResponse);
}
```

    4. proto/common/common.proto

```
syntax = "proto3";

package common;

option ...
option ...

enum Genre {
    DRAMA = 0;
    ACTION = 1;
    THRILLER = 2;
    WESTERN = 3;
}
```

Spring Boot Integration - User Service - Implementation

1. `com.grpcflix.user.UserApplication.java`

```
@SpringBootApplication
public class UserApplication {
    public static void main(String[] args) {
        SpringApplication.run(UserApplication.class, args);
    }
}
```

2. `com.grpcflix.service.UserService.java`

```
@GrpcService // automatically adds this service to the Grpc server when it starts
public class UserService extends UserServiceGrpc.UserServiceImplBase {
```

```
    @Autowired
    private UserRepository repository;

    @Override
    public void getUserGenre(UserSearchRequest request, StreamObserver<UserResponse> responseObserver) {
        UserResponse.Builder builder = UserResponse.newBuilder();

        this.repository.findById(request.getLoginId())
                .ifPresent(user -> {
                    builder.setName(user.getName())
                        .setLoginId(user.getLogin())
                        .setGenre(Genre.valueOf(user.getGenre().toUpperCase()))
                });
        responseObserver.onNext(builder.build());
        responseObserver.onCompleted();
    }

    @Override
    @Transactional
    public void updateUserGenre(UserGenreUpdateRequest request, StreamObserver<UserResponse> responseObserver) {
        UserResponse.Builder builder = UserResponse.newBuilder();

        this.repository.findById(request.getLoginId())
            .ifPresent(user -> {
                user.setGenre(request.getGenre().toString());
                builder.setName(user.getName())
                        .setLoginId(user.getLogin())
                        .setGenre(Genre.valueOf(user.getGenre().toUpperCase()));
            });

        responseObserver.onNext(builder.build());
        responseObserver.onCompleted();
    }
}
```

3. com.grpcflix.entity.User.java

```
@Data
@Entity
@ToString
public class User {
    @Id
    private String login;
    private String name;
    private String genre;
}
```

4. com.grpcflix.repository.UserRepository.java

```
@Repository
public interface UserRepository extends JpaRepository<User, String> {
}
```

Spring Boot Integration - User Service - Resources

1. UserService: src/main/resources/application.properties

```
grpc.server.port=6565
```

2. UserService: src/main/resources/data.sql

```
DROP TABLE IF EXISTS user;

CREATE TABLE user AS SELECT * FROM CSVREAD('classpath:user.csv');
```

3. resources/user.csv

```
login,name,genre
vinsguru,vinoth,WESTERN
sam123,sam,ACTION
```

4. Start UserApplication.java

[Breaking Spring/H2 Changes] - User Service

1. Recent breaking changes
    1. Use latest version of Spring Boot
    2. Use Java version 11 or above
    3. user-service/pom.xml
        1. grpc-server-spring-boot-starter: 2.13.1
    4. data.sql
        1. We cannot user `user` - change it to `customer`
            1. Or else use double quotes: "user"

```
DROP TABLE IF EXISTS "user";
CREATE TABLE "user" AS AS SEELCT * FROM ...;
```

    5. User.java

```
@Table(name = "\"user\"")
public class User {
    // ...
}
```

6. application.properties

```
spring.jpa.defer-datasource-initialization=true
```

1. Used to initialize using sql file

Spring Boot Integration - Movie Service - Implementation

1. com.grpcflix.movie.MovieApplication.java

```
@SpringBootApplication
public class MovieApplication {
    public static void main(String[] args) {
        SpringApplication.run(MovieApplication.class, args);
    }
}
```

2. movie.entity.Movie.java

```
@Data
@Entity
@ToString
public class Movie {
    @Id
    private int id;

    private String title;
    private int year;
    private double rating;
    private String genre;
}
```

3. movie.repository.MovieRepository.java

```
@Repository
public interface MovieRepository extends JpaRepository<Movie, Integer> {
    List<Movie> getMovieByGenreOrderByYearDesc(String genre);
}
```

4. movie.service.MovieService.java

```
@GrpcService
public class MovieService extends MovieServiceGrpc.MovieServiceImplBase {

    @Autowired
    private MovieRepsository repository;

    @Override
    public void getMovies(MovieSearchRequest request, StreamObserver<MovieSearchResponse> responseObserver) {
        List<MovieDto> movieDtoList = this.repository.getMovieByGenreOrderByYearDesc(request.getGenre().toString())
            .stream()
            .map(movie -> MovieDto.newBuilder()
                            .setTitle(movie.getTitle())
                            .setYear(movie.getYear())
                            .setRating(movie.getRating())
                            .build())
                            .collect(Collectors.toList());

        responseObserver.onNext(MovieSearchResponse.newBuilder().addAllMovie(movieDtoList).build());
        responseObserver.onCompleted();
    }

}
```

Spring Boot Integration - Movie Service - Resources

1. movie-service/../resources

1. application.properties

```
grpc.server.port=7575
```

2. data.sql

```
DROP TABLE IF EXISTS movie;

CREATE TABLE movie AS SELECT * FROM CSVREAD('classpath:movie.csv');
```

3. movie.csv

```
id, title, year, rating, genre
1, The Shawshank Redemption, 1994, 9.3, DRAMA
2, The godfather, 1972, 9.2, DRAMA
3, The Message, 1976, 10, ACTION
```

2. Run the application

[Breaking Spring/H2 Changes] - Movie Service

1. Use latest version of:

1. grpc-server-spring-boot-starter

1. 2.13.1.RELEASE

2. movie.csv

```
id, title, release_year, rating, genre
```

1. We cannot use `year` (keyword)

2. Movie.java

```
private int releaseYear;
```

1. Or:

```
@ColumnName(name="release_year")
private int year;
```

3. application.properties

```
spring.jpa.defer-datasource-initialization=true
```

Spring Boot Integration - Aggregator Service - Controller

1. `src/main/java:com.grpcflix.aggregator.AggregatorApplication.java`

```
@SpringBootApplication
public class AggregatorApplication {
    public static void main(String[] args) {
        SpringApplication.run(AggregatorApplication.class, args);
    }
}
```

2. `aggregator.dto.RecommendedMovie.java`

```
@Data
@AllArgsConstructor
@NoArgsConstructor
public class RecommendedMovie {
    private String title;
    private int year;
    private double rating;
}
```

3. `aggregator.dto.UserGenre.java`

```
@Data
public class UserGenre {
    private String loginId;
    private String genre;
}
```

4. `aggregator.controller.AggregatorController.java`

```
@RestController
public class AggregatorController {
    @GetMapping("/user/{loginId}")
    public List<RecommendedMovie> getMovies(@PathVariable String loginId) {

    }

    @PutMapping("/user")
    public void setUserGenre(@RequestBody UserGenre) {

    }
}
```

Spring Boot Integration - User Movie Suggestion Service - Part 1

1. `aggregator.service.UserMovieService.java`

```
@Service
public class UserMovieService {
    @GrpcClient("user-service")
    private UserServiceGrpc.UserServiceBlockingStub userStub;

    @GrpcClient("movie-service")
    private MovieServiceGrpc.MovieServiceBlockingStub movieStub;

    public List<RecommendedMovie> getUserMovieSuggestions(String loginId) {
        UserSearchRequest userSearchRequest = UserSearchRequest.newBuilder()
            .setLoginId(loginId)
            .build();
        UserResponse userResponse = userStub.getUserGenre(userSearchRequest);

        MovieSearchRequest movieSearchRequest = MovieSearchRequest.newBuilder()
            .setGenre(userResponse.getGenre())
            .build();

        MovieSearchResponse movieSearchResponse = movieStub.getMovies(movieSearchRequest);

        return movieSearchResponse.getMovieList()
            .stream()
            .map(movieDto -> new RecommendedMovie(movieDto.getTitle(),
                movieDto.getYear(),
                movieDto.getRating()
            )
            .collect(Collectors.toList());
    }
}
```

Spring Boot Integration - User Movie Suggestion Service - Part 2

1. `aggregator.service.UserMovieService.java`

```
public void setUserGenre(UserGenre userGenre) {
    UserGenreUpdateRequest userGenreUpdateRequest = UserGenreUpdateRequest.newBuilder()
        .setLoginId(userGenre.getLoginId())
        .setGenre(Genre.valueOf(userGenre.getGenre().toUpperCase()))
        .build();

    UserResponse userResponse = this.userStub.updateUserGenre(userGenreUpdateRequest)
}
```

2. `aggregator.controller.AggregatorApplication.java`

```
@Autowired
private UserMovieService userMovieService;

... getMovie(...) {
    return this.userMovieService.getUserMovieSuggestions(loginId);
}

... setUserGenre(...) {
    this.userMovieService.setUserGenre(userGenre);
}
```

3. `resources/application.yaml`

```
grpc:
    client:
        user-service:
            address: static://localhost:6565
            negotiationType: plaintext
        movie-service:
            address: static://localhost:7575
            negotiationType: plaintext
```

Spring Boot Integration - User Movie Suggestion Service - Demo

1. Start all services:
    1. AggregatorApplication
        1. Connects to MovieService and UserService only when we make the first RPC call
            1. Even if the dependent services are down, it will start
    2. MovieApplication
    3. UserApplication
2. PostMan

```
GET: localhost:8080/vinsguru
PUT: localhost:8080/user
```

Spring Boot With gRPC + SSL / TLS

1. Steps to enable SSL/TLS in Spring Boot:
    1. Server Side Changes:
        1. Add the properties:

```
grpc.server.security.enabled=true
grpc.server.security.certificateChain=file:/home/vinsguru/ssl-tls/localhost.crt
grpc.server.security.privateKey=file:/home/vinsguru/ssl-tls/localhost.pem
```

    2. Client Side Changes:
        1. When client talks to multiple servers, if any of the servers is using SSL/TLS, then client `negotiationType` must be set to TLS

```
grpc:
    client:
        user-service:
            address: static://localhost:6565
            negotiationType: plaintext
        movie-service:
            address: static://localhost:7575
            negotiationType: TLS
            security:
                trustCertCollection: file:/home/vinsguru/ssl-tls/ca.cert.pem
```

Section 15: What's Next

What's Next?

1. Mocroservices:
    1. Processing many requests:
        1. Options to develop a microservice:
            1. Spring Web + Java Virtual Thread
                1. Synchronous blocking style code but Java does non-blocking I/O for us to process requests more efficiently
            2. Reactive Microservices with Spring WebFlux
                1. Knowledge on Reactive programming is required
                    1. Stream-based communication
    2. Backend-communication:
        1. Network problems

        1. Options
            1. gRPC
                1. Google
            2. RSocket (reactive)
                1. Netflix
            3. Kafka (event-driven async)
                1. LinkedIn
    2. Request-Response style
        1. Client will send a request
        2. Server will send a response
        3. Options:
            1. Spring Web - Sync/REST (no stream based)
            2. Spring WebFlux - Reactive/REST (request/response, stream based)
            3. gRPC - Sync/**Low Latency**/Bi-directional streaming
                1. Network Layer 7 is used
            4. rSocket - Reactive/**Low Latency**/Bi-directional streaming
                1. Network Layer 5 is used
                2. **Better performance than gRPC**
                    1. However, gRPC is not tied to HTTP/2
            5. Kafka - Event-Driven Async
    3. Stream-based communication
        1. Continuous exchange of information simultaneously
3. Frontend-communication:
    1. GraphQL
        1. Efficient data retrieval
    2. gRPC & rSocket - for mobile apps
4. Caching:
    1. Redis
        1. Super-fast read/write operations
            1. Improves application throughput
5. Resiliency:
    1. Design patterns to develop resilient applications
        1. **Integration and Resilient Design Patterns for Microservices Architecture**
6. Regression Testing
    1. Selenium
    2. Jenkins
7. Deployment & Orchestration
    1. Docker
        1. Packaging the application with all the dependencies
        2. Consistent development and deployment
    2. Kubernetes
        1. Application deployment
        2. Load balancing
        3. Service discovery
        4. Auto scaling
        5. Config management
        6. Avoid vendor lock-in/ Multi cloud support like AWS/ GCP/ Azure...